

Stochast — Methods Plugin Manual

Companion to *Stochast: Statistically Grounded Generative Modules for VCV Rack*

May 2026

About this manual

This manual is the practical companion to the *Stochast* Methods plugin: a fifteen-module statistical layer for VCV Rack 2. It documents every parameter, input, output, light, and right-click context-menu option for every Methods module, along with the installation steps, the conventions shared across the suite, and the reference patches distributed with the source.

Each module is a real statistical process — a sampler, an estimator, a hypothesis test, the bootstrap — turned into knobs and control voltage you can patch as generative, modulation, or sequencing material. The accompanying technical notes cover the numerics and the analog-computer lineage; this manual stays close to the keys, knobs, and jacks. (For *teaching* statistics, reach for the browser-based Empiria web app at <https://kevinschoenholzer.com/empiria/> instead.)

The wider Stochast suite includes four additional plugins (**Polis**, **Epi**, **Space**, **Decisions**) covering agent-based social models, network epidemiology, spatial dynamics, and behavioral decision-making. They share the Methods panel grammar and patch conventions, but their per-module documentation is the subject of a separate forthcoming companion manual; references to them in this text are kept brief.

Quick start

Installing VCV Rack

1. Download VCV Rack 2 from <https://vcvrack.com/Rack>. The Free edition is sufficient for everything in this manual; the Pro edition is needed only if you want to run Stochast inside another DAW as an audio-plug-in host.
2. Install on macOS, Windows, or Linux.
3. Launch Rack at least once so it creates the plugin folder at:
 - macOS: `~/Library/Application Support/Rack2/plugins-<arch>/`
 - Windows: `%LOCALAPPDATA%\Rack2\plugins-win-x64\`
 - Linux: `~/.Rack2/plugins-lin-x64/`

Installing the Stochast Methods plugin

Either:

- **Pre-built .vcvplugin archive** (recommended): download the `SHLabs-Methods-<version>-<platform>.vcvplugin` file from the repository release page, then drop it into the plugin folder above. Restart Rack; the modules will appear in the module browser under the **SHLabs** brand.
- **Build from source** (developers only): clone the repository, set `RACK_DIR` to point at the Rack SDK, and run `make dist` in the `Methods/` subdirectory. The build produces a .vcvplugin archive in `dist/`. Copy it into the plugin folder above and restart Rack.

The Methods plugin has no dependencies beyond VCV Rack 2. The binary size is well under 1 MB.

A 60-second tour

1. Launch Rack. You start with an empty rack and the **Audio** and **Audio-8** I/O modules.
2. Right-click any empty rack space to open the module browser.
3. Type “SH Labs” in the search field. The fourteen Methods modules appear with their color-stripe family marker (cyan band under the header).
4. Drop **Sample** and **Frame** onto the rack, side-by-side.
5. Right-click  “What does this do?” on either module to read a one-paragraph summary.
6. Drag a cable from **Sample**’s MEAN output to **Frame**’s SIG input.
7. Press the **Play** button (top of the Rack window) to start audio processing. **Sample** begins drawing observations from a Normal distribution; **Frame** accumulates them and displays the running histogram with a vertical mean line and a shaded confidence-interval band.

You are now running a pedagogical pipeline that would have taken roughly twenty lines of R to set up. The next sections describe how to extend it.

Conventions shared across Methods modules

Panel grammar

Every Methods module follows the same vertical layout, so once a student has read one module they can read any other:

```
+-----+
|  MODULE NAME  |  header strip (drawn in NanoVG)
+-----+
|               |
|  visualization |  live state - histogram, scatter, etc.
|               |
+-----+
|               |  primary knob row (1-4 knobs)
+-----+
|  trim / SHUFFLE |  secondary row (toggles, buttons)
+-----+
|  IN             |  input jack row (up to 4)
+-----+
|  OUT            |  output jack row (up to 4)
+-----+
```

The viz area is always at the same height regardless of module, and the jack rows are always at the bottom, so a learner can patch by muscle memory.

Clocks, resets, polyphony

Most analytical modules carry **CLOCK** and **RESET** inputs at the left of the input row:

- **CLOCK** accepts any rising-edge trigger or gate as the tick signal. If left unpatched, the module advances at its own internal rate (typically 30 Hz). Patching a clock makes the pipeline sample-rate-deterministic — see the **Seed** section on reproducibility.
- **RESET** clears the module’s internal buffer / re-initializes its state. Patching the **Seed** module’s TRIG output into every module’s RESET input lets you re-initialize an entire pipeline in lockstep with one button press.

Cables in VCV Rack are polyphonic by default — a single cable can carry up to 16 voices. Most Methods modules support polyphonic input transparently: every channel contributes one observation per clock tick, so the same pipeline scales from one to sixteen parallel pseudo-experiments without adding modules.

Reproducibility primitive

Every random draw inside Stochast is produced by a Mersenne-Twister RNG (std: :mt19937, period $2^{19937} - 1$) seeded explicitly at construction. The **Seed** module exposes an integer seed (0–999) as a control voltage and as a change-trigger pulse. Connect the trigger to every random module's RESET input and the patch becomes byte-identical across machines: the same seed produces the same observations, the same t -statistic, the same bootstrap interval.

Module reference

The fifteen modules below follow the canonical empirical workflow left-to-right: data-generating process → sampling → estimation → inference → diagnostics → support utilities. Each entry lists parameters, inputs, outputs, lights, the live visualization, and the right-click context-menu options. The format is consistent across modules.

Sample

A parametric data-generating process. Draws an i.i.d. stream from a user-selectable distribution at clock rate and reports running descriptives.

Param	Range	Default	Effect
DIST	snap 0..3	Normal	Normal / Uniform / Exponential / Beta
P1	0..1 normalized	0.50	First parameter (μ / center / λ / α)
P2	0..1 normalized	0.30	Second parameter (σ / width / $-\beta$)
WIN	4..1024	128	Window for running mean / SD
SHUFFLE	button	—	Re-seed RNG and clear window

Input	Type	Notes
CLOCK	trig	Draws one sample per rising edge; ~30 Hz internal if unpatched
RESET	trig	Clear window and re-seed
P1·CV	CV	±5 V adds to P1 (post-normalization)
P2·CV	CV	±5 V adds to P2

Output	Range	Notes
SAMPLE	± 12 V	Most recent draw
MEAN	± 12 V	Running mean over WIN
SD	0..12 V	Running SD over WIN
TRIG	gate	2 ms pulse on each new sample

Visualization. Empirical histogram of the last WIN samples overlaid on the theoretical PDF (rendered in gold). As n grows the bars converge to the curve — useful for demonstrating that a named distribution is *not* an abstract object but a long-run frequency.

Right-click presets. Under *Real-world distribution presets* the menu offers six unit-relabellings that move the same parametric draws onto substantive scales: adult height (Normal-like), IQ scores (Normal $\mu = 100$, $\sigma = 15$), reaction times (Exponential), survey responses (Uniform on $[-1, 1]$), U-shaped opinion (Beta $\alpha = \beta = 0.2$), right-skewed income (Beta $\alpha = 1$, $\beta = 4$), near-Bernoulli coin flip (Beta $\alpha = \beta = 0.05$), and bell-shaped narrow (Normal $\sigma = 0.3$). Each preset just sets P1 and P2 — no new mathematics; the same draws acquire a new substantive label.

Implementation. `std::mt19937 + <random>` distributions for Normal / Uniform / Exponential; an inverse-CDF sampler for Beta to avoid the rejection-method's tail behavior at extreme α , β .

Frame

The sampling-frame measurement window. Collects samples on a clock into a buffer of configurable length and reports the sample mean, sample SD, and standard error of the mean.

Param	Range	Default	Effect
MODE	snap 0..2	Running	SNAPSHOT / RUNNING / GROWING
N	4..4096	128	Buffer size
CI	snap 0..3	95 %	Confidence level (80 / 90 / 95 / 99)
SUB	1..16	1	Take every Kth clock tick
CLEAR	button	—	Clear buffer

Input	Type	Notes
CLOCK	trig	~30 Hz internal if unpatched
RESET	trig	Clear buffer
SIG	CV	Signal to measure (polyphonic; every channel contributes one observation)
TRIG	trig	Manual one-shot snapshot in SNAPSHOT mode

Output	Range	Notes
MEAN	± 12 V	Sample mean (large readout on top-right of viz)
SD	0..12 V	Unbiased sample SD
SE	0..12 V	Standard error = $SD / \sqrt{n}$

Output	Range	Notes
READY	gate	SNAPSHOT: high after buffer is full; RUN/GROW: pulses on each new sample

Visualization. Buffer histogram with a vertical mean line and a shaded confidence-interval band. Mode names appear top-left; mean and SE appear top-right at large font. As more samples accumulate in GROWING mode the band visibly contracts as $1 / \sqrt{n}$ — the Law of Large Numbers made visible.

Modes.

- **SNAPSHOT** — collect N , then freeze. The view is cross-sectional; further clock ticks are ignored until RESET. Useful when the question is “what does *one* sample of size n look like?”
- **RUNNING** — ring buffer of the latest N . Old samples are overwritten by new ones. The mean and CI band are continuously updated. Useful for tracking a non-stationary signal.
- **GROWING** — accumulator. Samples are added without bound until the maximum buffer size is reached. The CI band shrinks as $1 / \sqrt{n}$ on screen — the canonical LLN demonstration.

Regress

Online ordinary-least-squares regression of Y on X . Pairs (X, Y) are sampled on each clock tick and stored in a buffer; the line $Y = \alpha + \beta \cdot X$ is fit by minimising sum of squared residuals.

Param	Range	Default	Effect
MODE	snap 0..2	Running	SNAPSHOT / RUNNING / GROWING
N	4..2048	128	Buffer size
CI	snap 0..3	95 %	Confidence level for the band
SUB	1..16	1	Sub-sample factor
CLEAR	button	—	Clear buffer

Input	Type	Notes
CLOCK	trig	~30 Hz internal
RESET	trig	Clear buffer
X	CV	Predictor (polyphonic OK)
Y	CV	Response (polyphonic OK)

Output	Range	Notes
β	± 12 V	Slope estimate (large readout on top-right of viz)
α	± 12 V	Intercept estimate
R^2	0..10 V	Coefficient of determination
RESID	± 12 V	$Y - (\hat{\alpha} + \hat{\beta} \cdot X)$ on channel 1

Visualization. Scatter of the buffer’s (X, Y) pairs with the most recent observations highlighted, the fitted line drawn in gold, and a shaded confidence band whose width is $z \cdot \sqrt{(\text{MSE} \cdot (1/n + (x - \bar{x})^2 / \sum(x - \bar{x})^2))}$. The band is the classical “trumpet” shape — narrow at the center of the X range, widening at the extremes — and is clipped to the chart area so it never extends outside the viz frame.

Implementation. Online sufficient-statistic update (Welford’s algorithm) so β , α , R^2 , and MSE recompute in $O(1)$ per tick.

Test

One-sample or two-sample Welch's t -test against $H_0: \mu = \mu_0$ (one-sample) or $H_0: \mu_1 - \mu_2 = \delta_0$ (two-sample, unequal-variance Welch's t).

Param	Range	Default	Effect
MODE	snap 0..2	Running	Buffer behavior
N	4..2048	128	Buffer size per group
α	snap 0..2	0.05	Significance level (0.01 / 0.05 / 0.10)
H_0	-5..+5 V	0.0	μ_0 (one-sample) or δ_0 (two-sample)
TEST	snap 0..1	One-sample	One- or two-sample (trimpot)
CLEAR	button	—	Clear buffers

Input	Type	Notes
CLOCK	trig	~30 Hz internal
RESET	trig	Clear buffers
SIG	CV	Group 1 (polyphonic OK)
SIG ₂	CV	Group 2 (used only in two-sample mode)

Output	Range	Notes
t	± 12 V	Observed t -statistic (large readout on top-right of viz)
p	0..10 V	Two-tailed p -value
REJ	gate	High when reject- H_0 at the current α
d	± 12 V	Cohen's d effect size

Visualization. t -distribution null with degrees of freedom matching the test, shaded rejection regions at the selected α level (both tails), the observed t marked as a vertical line, and a *REJECT* / *n.s.* indicator chip with significance stars on the p -value (□ for $p < 0.05$, □ □ for $p < 0.01$, □ □ □ for $p < 0.001$).

Implementation. Two-tailed p is computed *exactly* via the regularized incomplete beta function $I(x; a, b)$ using Lentz's continued-fraction expansion, following the standard numerical recipe (Press et al., 2007). The viz overlays the t -PDF matching the test's degrees of freedom, not a Gaussian approximation.

Boot

Non-parametric bootstrap resampling of a user-selected statistic. Given a buffer of n samples, draws B resamples with replacement and computes the statistic on each; the *empirical bootstrap distribution* of the estimator is plotted on the panel.

Param	Range	Default	Effect
MODE	snap 0..2	Snapshot	Buffer behavior

Param	Range	Default	Effect
N	4..2048	128	Sample size
B	50..2000	500	Bootstrap iterations
STAT	snap 0..3	Mean	Mean / Median / SD / Variance
CI	snap 0..3	95 %	Percentile-CI level
CLEAR	button	—	Clear buffer + re-seed RNG

Input	Type	Notes
CLOCK	trig	~30 Hz internal
RESET	trig	Clear buffer
SIG	CV	Signal to sample (polyphonic OK)
RSAMP	trig	Force a fresh bootstrap on the same data

Output	Range	Notes
EST	±12 V	Point estimate (statistic on the original sample)
SE	0..12 V	Bootstrap SE = SD of the bootstrap distribution
CI_LO	±12 V	Lower confidence bound
CI_HI	±12 V	Upper confidence bound

Visualization. Histogram of the B bootstrap statistics, with a vertical line at the point estimate, brackets at the percentile-CI bounds, and live readouts of the bias-correction z_0 and acceleration $\hat{a}$ (the two adjustments that turn a percentile CI into a BCa CI). Patching a clock into RSAMP fires a fresh bootstrap on each tick, so a student can watch the bootstrap's *own* variability against a frozen sample.

Implementation. Bias-corrected and accelerated (BCa) interval following Efron (1987). The z_0 correction is the inverse normal CDF of the proportion of bootstrap statistics less than the point estimate; $\hat{a}$ is the standard jackknife acceleration. Both move the interval away from the percentile baseline when the bootstrap distribution is skewed.

Lag

Time-series autocorrelation analysis. Estimates the autocorrelation function $\rho(k)$ at lags $1..K$ from a buffered signal, plus the AR(1) coefficient $\phi = \rho(1)$ and the AR(1) residual variance $\sigma^2\epsilon = \sigma^2(1 - \phi^2)$.

Param	Range	Default	Effect
MODE	snap 0..2	Running	Buffer behavior
N	4..2048	128	Buffer size
LAGS	2..32	16	Number of lags displayed and emitted
DETREND	snap 0..2	Off	Off / Mean / Linear (handles non-stationary input)
CLEAR	button	—	Clear buffer

Input	Type	Notes
CLOCK	trig	~30 Hz internal
RESET	trig	Clear buffer
SIG	CV	Signal to analyze (polyphonic OK)

Output	Range	Notes
ϕ	± 10 V	AR(1) coefficient = $\rho(1)$
ACF	poly	$\rho(1)..\rho(K)$ emitted as a polyphonic cable
$\sigma^2\epsilon$	0..12 V	Residual variance of the AR(1) fit
WHITE	gate	High when <i>all</i> displayed lags lie within the Bartlett band

Visualization. Stem plot of $\rho(k)$ for $k = 1..K$ with shaded $\pm 1.96 / \sqrt{n}$ Bartlett significance bands. Bars outside the band indicate significant autocorrelation at that lag.

Implementation. Biased sample autocorrelation $\hat{\rho}(k) = \sum (x_t - \bar{x})(x_{t+k} - \bar{x}) / \sum (x_t - \bar{x})^2$. The DETREND option controls whether the buffer's mean or a linear trend is removed before the estimate, which matters when the signal is non-stationary.

Code

Continuous $\rightarrow$ ordinal-categorical encoder. Maps an incoming CV stream into K ordinal categories (Likert-style 1.. K) by uniform cutpoints over the user-set range [LOW, HIGH]. Bridges continuous data-generating processes to survey-style categorical observations.

Param	Range	Default	Effect
K	2..7	5	Number of categories
LOW	-10..10	-5	Lower bound of the cutpoint range
HIGH	-10..10	+5	Upper bound of the cutpoint range
N	4..1024	128	Window for empirical category statistics
SHUFFLE	button	—	Clear empirical counts

Input	Type	Notes
CLOCK	trig	~30 Hz internal
RESET	trig	Clear counts
SIG	CV	Continuous signal to encode (polyphonic OK)

Output	Range	Notes
CAT	1.. K V	1 V per category, on the same polyphony as SIG
GATES	poly	One channel per category; high for the channel matching the current CAT
MEAN	1.. K V	Running mean category over N
ENT	0..12 V	Shannon entropy of the empirical category distribution (in bits)

Visualization. Bar chart of the empirical category proportions over the running window, plus the current observation marker.

Implementation. Cutpoints are uniformly spaced inside [LOW, HIGH]: bin i covers $[LOW + (i - 1) \cdot (HIGH - LOW) / K, LOW + i \cdot (HIGH - LOW) / K)$.

Tab

Contingency-table / cross-tabulation module. Two categorical CV streams (X , Y) — typically the outputs of two Code modules — are paired on each clock tick and tabulated into a $K_1 \times K_2$ frequency matrix. Reports the χ^2 statistic, Cramér's V effect size, and an INDEP gate.

Param	Range	Default	Effect
K1	2..7	5	Number of categories on the X axis
K2	2..7	5	Number of categories on the Y axis
LOW	-10..10	1	Lowest expected CV value (assumes Code-style 1 V/cat)
HIGH	-10..10	7	Highest expected CV value
CLEAR	button	—	Clear the frequency matrix

Input	Type	Notes
CLOCK	trig	~30 Hz internal
RESET	trig	Clear matrix
X	CV	First categorical stream (polyphonic OK)
Y	CV	Second categorical stream (polyphonic OK)

Output	Range	Notes
χ^2	0..12 V	χ^2 statistic
V	0..10 V	Cramér's V ($\in [0, 1]$; emitted $\times 10$)

Output	Range	Notes
INDEP	gate	High when $V < 0.1$ (effective independence)
N	0..10 V	Total observation count (emitted $\div$ 100)

Visualization. Heatmap of the frequency matrix with cell shading proportional to relative frequency. Row and column marginals are shown as faint bars on the right and bottom.

Implementation. Exact p -values use the regularized incomplete gamma function — same numerical recipe as Test’s incomplete beta. Cramér’s $V = \sqrt{\chi^2 / (n \cdot (\min(K_1, K_2) - 1))}$.

Strata

Real-time STL-style decomposition. Splits an incoming CV stream into trend (low-pass drift), seasonal (period-locked recurring component), and residual (everything else) signals on three separate outputs.

Param	Range	Default	Effect
TREND	0..1	0.35	Trend cut-off (lower = smoother trend)
PERIOD	0..1	0.40	Seasonal period (logarithmic, ~10..1000 samples)
MEMORY	0..1	0.50	Seasonal-template memory (lower = faster adaptation)

Input	Type	Notes
PERIOD·CV	CV	$\pm 5 \text{ V} \times 0.25.. \times 4$ multiplier on the period knob
SIG	CV	Signal to decompose

Output	Range	Notes
TREND	$\pm 12 \text{ V}$	Low-pass component
SEASONAL	$\pm 12 \text{ V}$	Period-locked recurring component
RESIDUAL	$\pm 12 \text{ V}$	Whatever the trend and seasonal don’t explain

Visualization. Three stacked traces — TREND on top in cyan, SEASONAL in amber, RESIDUAL in red — sharing a common time axis inside the viz area.

Implementation. A one-pole low-pass extracts the trend; the seasonal template is a running average over the configured period using exponential decay (the MEMORY knob); the residual is the input minus trend minus seasonal.

Cohort

Online k -means quantizer. Learns K cluster centers from incoming CV and snaps the output to the nearest center. A data-driven alternative to fixed *Code* cutpoints.

Param	Range	Default	Effect
K	2..12	6	Number of cluster centers (integer)
RATE	0..1	0.30	Learning rate (higher = faster center updates)
RANGE	0..1	0.50	Reset spread (initial center dispersion, $\pm V$)
SEMI	switch	Off	Snap output to nearest semitone (V/oct) for pitched applications
RESET	button	—	Re-initialize cluster centers

Input	Type	Notes
SIG TRIG	CV trig	Signal to quantise Update trigger (centers update continuously if TRIG is unpatched)

Output	Range	Notes
OUT	± 10 V	The nearest cluster center to the current SIG value

Light	Notes
SEMI	Lit when the SEMI switch is engaged
RESET	Briefly flashes on a RESET event

Visualization. A vertical strip showing the current input value as a moving marker against the K cluster centers rendered as horizontal ticks. When centers are still adapting they drift on the strip; when learning is converged they sit still.

Implementation. Each tick, the nearest center is updated by $\text{RATE} \cdot (\text{sample} - \text{center})$. On RESET, centers are spread uniformly across $[-\text{RANGE} \cdot 5 \text{ V}, +\text{RANGE} \cdot 5 \text{ V}]$. The semitone-snap option post-processes OUT through a V/oct quantizer to give pitched musical applications.

Factor

Online principal-component analysis on six CV inputs via Sanger's generalised Hebbian rule. Outputs the top three latent components as control voltages — a real-time factor-analysis mixer for modulation.

Param	Range	Default	Effect
RATE	0..1	0.35	Learning rate η for the Hebbian update
SCALE	0..1	0.50	Output gain on the PC outputs (logarithmic)

Param	Range	Default	Effect
CENTER	switch	On	Subtract running mean before extracting PCs
FREEZE	switch	Off	Stop learning; keep the current basis fixed
RESET	button	—	Re-initialize the basis to identity

Input	Type	Notes
A1..A6	CV	Six independent input channels

Output	Range	Notes
PC1	±10 V	Projection onto the first principal component
PC2	±10 V	Projection onto the second principal component
PC3	±10 V	Projection onto the third principal component

Light	Notes
CENTER	Lit when centering is active
FREEZE	Lit when learning is frozen
RESET	Briefly flashes on RESET

Visualization. Left half: PC1-vs-PC2 scatter with a fading trail showing the latent trajectory. Right half: bar plot of the six absolute loadings $|W[k][n]|$ for each of the three components, color-coded by PC (yellow / cyan / magenta).

Implementation. Sanger's rule $\Delta w_j = \eta \cdot y_j \cdot (x_{\text{centred}} - \sum_{\{j \leq i\}} y_j \cdot w_j)$, followed by per-row L2 normalization for numerical stability. PC outputs are clamped to ± 10 V at storage so the scatter can never leak outside the viz frame even at high SCALE.

Seed

Reproducibility primitive. Displays a single integer seed value prominently (0..999) and emits it as both a CV and a change-trigger pulse. Patch the trigger into every random module's RESET input so that the whole pipeline re-initializes in lockstep when the seed changes.

Param	Range	Default	Effect
VALUE	0..999	42	The seed value (integer, snap)
RANDOMIZE	button	—	Pick a fresh seed uniformly at random

Input	Type	Notes
CLOCK	trig	Increments VALUE by 1 on each tick
RESET	trig	Sets VALUE back to its saved patch value

Output	Range	Notes
SEED·CV	0..10 V	VALUE / 100 (so 999 $\approx$ ~10 V)
TRIG	gate	2 ms pulse on every VALUE change

Visualization. Large 0–999 numeric readout of the current VALUE, with a thin progress bar at the bottom showing VALUE / 999.

Reproducibility note. The seed VALUE is persisted in the patch JSON, so a .vcv file shared between machines opens with the same seed and therefore produces the same simulation. The TRIG output is the recommended way to fan a reset out to many modules; the SEED·CV output is useful when a downstream module wants to *use* the seed as a parameter (e.g. as a Beta's α via a CV input).

Tape

Polyphonic record-and-replay buffer for CV streams. Three modes toggle between recording the input, playing the buffer once, and looping it indefinitely. Pairs with the right-click CSV *export* item for taking a buffered dataset out of Stochast into R / Python.

Param	Range	Default	Effect
MODE	snap 0..2	REC	REC / PLAY / LOOP
LENGTH	4..4096	1024	Buffer length (samples)
SPEED	0..2	1.0	Playback speed ($\times 0.5$.. $\times 2$)
CLEAR	button	—	Clear buffer

Input	Type	Notes
CLOCK	trig	~30 Hz internal
RESET	trig	Restart playback at index 0
SIG	CV	Signal to record (polyphonic; each channel becomes its own track)
TRIG	trig	Manual one-shot — REC: insert one sample; PLAY: play one sample

Output	Range	Notes
SIG	± 12 V	Replayed signal (polyphonic, channels preserved)
POS	0..10 V	Playback cursor position (0 V = start, 10 V = end of buffer)
WRAP	gate	2 ms pulse when the cursor wraps at end of buffer
ACT	gate	High while recording or playing

Right-click  “**Export buffer to CSV...**” opens a file dialog and writes the buffer to a plain CSV with one row per recorded sample and one column per polyphonic channel. Format:

```
sample,ch1,ch2,...,chN
0, <ch1 value>, <ch2 value>, ...
1, <ch1 value>, <ch2 value>, ...
...
```

Loadable in R (`read.csv("tape.csv")`) or Python (`pandas.read_csv("tape.csv")`).

Gauge

Translates a control voltage into a real-world quantity using a linear mapping $x = A \cdot V + B$. The two knobs drive the mapping at all times; right-click presets stamp known (A, B) pairs into the knobs for substantive units.

Param	Range	Default	Effect
A	−200..+200	1.0	Slope (multiplier)
B	−2000..+2000	0.0	Offset
DECIM	0..4	2	Display decimal places
RANGE	0..1	1.0	Bar range hint (0 = narrow, 1 = wide)

Input	Type	Notes
IN	CV	Signal to convert (polyphonic; readout shows channel 1)

Output	Range	Notes
VAL	±12 V	Converted value as CV
THRU	±12 V	Pass-through of IN, unchanged (for chaining)

Visualization. Large numeric readout of $A \cdot V + B$ on the first input channel; unit label below; recent-range bar at the bottom shows where the value sits within the preset’s typical span. A modified-from-preset hint turns the preset name amber when the user has dialed the knobs away from the preset’s stamped values.

Right-click presets. The preset menu stamps a known (A, B) pair into the knobs:

Preset	(A, B)	Input range	Reads as
Voltage (passthrough)	(1, 0)	any V	raw V
Percent — unipolar	(10, 0)	0..10 V	0..100 %
Percent — bipolar	(10, 50)	−5..+5 V	0..100 %
Percent — signed	(10, 0)	−10..+10 V	±100 %
Probability	(0.1, 0)	0..10 V	0..1
Z-score	(1, 0)	±3 V	z-score
IQ ($\mu = 100, \sigma = 15$)	(15, 100)	±3 V	IQ score
Adult height (cm)	(10, 170)	±4 V	130..210 cm
Likert 5-point	(0.4, 3)	±5 V	1..5
Likert 7-point	(0.6, 4)	±5 V	1..7

Preset	(A, B)	Input range	Reads as
Test score (SAT)	(100, 500)	±3 V	200..800
Temperature °C	(5, 20)	±10 V	−30..70 °C
Temperature °F	(9, 68)	±10 V	−22..158 °F
Reaction time (ms)	(500, 0)	0..6 V	0..3000 ms
Count	(10, 0)	0..10 V	0..100
Custom	(1, 0)	—	identity reset

After stamping a preset, the user may tweak *A* and *B* by hand; the preset name then reads (*modified*) in amber. The *Snap knobs to current preset* menu item reverts manual edits.

Quantity

The mirror of Gauge for the input side of a pipeline. Where Gauge reads a downstream CV and displays it as a real-world unit (“IQ = 115”), *Quantity* lets the user *dial* a real-world value directly and emits the back-converted CV:

$$CV = (VALUE - B) / A$$

Selecting a preset stamps a known (*A*, *B*, *VALUE*) triple into the knobs, so a learner can set Test’s H_0 as “IQ = 100” or Code’s LOW cutpoint as “−2 z-scores” without mentally inverting the mapping.

Param	Range	Default	Effect
VALUE	−2000..+2000	0.0	The real-world value in current preset units
A	−200..+200	1.0	Slope (same as Gauge)
B	−2000..+2000	0.0	Offset (same as Gauge)
DECIM	0..4	2	Display decimal places

Input	Type	Notes
CV	CV	Modulates VALUE in real-world units (+1 V ⇔ +1 unit of the active preset)

Output	Range	Notes
CV	±12 V	The back-converted CV: $(VALUE - B) / A$, clamped to ±12 V
THRU	±12 V	Pass-through of the CV input, unchanged (for chaining)

Visualization. Same layout as Gauge: preset name at the top (turns amber and reads (*modified*) when the knobs no longer match the preset’s defaults), big numeric readout of VALUE, unit label below, a line showing the emitted CV, and the active $V = (VALUE - B) / A$ formula reminder at the bottom.

Right-click presets. Same sixteen presets as Gauge. Selecting one stamps the preset's (A , B , $VALUE$) triple into the knobs. The *Snap knobs to current preset* menu item reverts manual edits.

Typical pairing. *Quantity* sits on the input side of a pipeline (driving *Test*'s H_0 , *Code*'s LOW / HIGH cutpoints, a *Sample*'s P1 / P2 via the CV inputs, etc.) and *Gauge* sits on the output side reading whatever quantity comes out the other end — a symmetric “*Quantity sets, Gauge reads*” division of labour.

Edge case. When $A = 0$ the back-conversion is undefined; the panel shows CV ? ($A = 0$) in red, and the CV output emits 0 V. This only happens if the user dials A to zero deliberately.

Worked examples

Three starter patches ship under `patches/` and reproduce three canonical pedagogical workflows. Each opens with a fixed seed so the figure on a student's screen matches the figure in the manual.

Visible Law of Large Numbers — `patches/empiria_screenshot_frame.vcv`

Seed (42) □ Sample (Normal, $\mu = 0$, $\sigma = 1$) □ Frame (GROWING). Press play and watch the empirical histogram fill in toward the theoretical Normal PDF, the mean readout converge to 0, and the CI band contract as $1 / \sqrt{n}$. After ~30 s of run-time, $n \approx 4000$ and $SE \approx 0.005$.

Seeded two-sample t -test — `patches/empiria_seeded_t_test.vcv`

Seed (42) □ two independent Sample modules □ Frame + Test (two-sample mode). The two Samples can be configured with different μ to produce a true effect, or identical μ to produce a null. Because the seed is fixed, t and p are reproducible across machines.

Coin-flip Law of Large Numbers — `patches/empiria_coin_flip_lln.vcv`

Seed □ Sample (Beta $\alpha = \beta \approx 0.05$ □ near-Bernoulli) □ Frame (GROWING) + Boot. The empirical mean must converge to the closed-form $p = 0.5$ as n grows; the bootstrap CI provides a visual reminder that the convergence is *probabilistic*, not monotone. Gauge, with the Probability preset, reinterprets the running mean as a 0..1 quantity for legibility.

Generating per-student variants

The bundled `tools/empiria_patch_gen.py` script regenerates any of the patches with a fresh seed:

```
python3 tools/empiria_patch_gen.py --seed 137 \
                                   --out handouts/student_137/
```

Wrap in a shell loop to mint one patch per student:

```
for s in 101 102 103 104 105 ; do
    python3 tools/empiria_patch_gen.py --seed "$s" \
                                       --out "handouts/student_$$/"
done
```

The same Python entry-points (`build_patch`, `write_patch`) are importable as a library for custom course material.

Documenting and exporting results

Screenshots

VCV Rack has a built-in screenshot facility:

- **View ▢ Screenshot** (Cmd/Ctrl-Shift-S) captures the full rack to a PNG file at the current zoom. Zoom in first if you want a high-resolution figure.
- Right-clicking inside any individual module exposes a *Screenshot* item that captures just that module's panel.

Saving the patch

A patch is plain JSON (typically 5–50 KB). Use **File ▢ Save As** to write a `.vcv` file. The file is human-readable, fully self-describing, and reproducible across machines — a collaborator who opens it sees exactly the same modules in the same positions with the same parameter values and the same seeded RNG state.

CSV export of buffered data (Tape)

See the **Tape** section above. The exported CSV is immediately loadable in R and Python; an example column header is `sample,ch1,ch2,...` with one row per recorded tick.

Composability with R / Python / Julia

Stochast is a *companion* to the standard statistical toolchain, not a replacement. The CSV export from Tape lets a student take a synthetic dataset they generated *inside* Stochast and analyze it *outside* in any environment of their choosing — useful both for assignments that require a specific software package and for graduate work that mixes Stochast's live visualization with an established analytic stack. The recommended workflow is:

1. Build a data-generating pipeline (`Sample` → Code for categorical responses, `Sample` → `Regress` for OLS, etc.).
2. Patch the output of interest into Tape's SIG input.
3. Switch Tape to REC and let it fill its buffer.
4. Right-click Tape ▢ *Export buffer to CSV* and choose a path.
5. Open the CSV in R / Python / Julia / Excel and carry the analysis further with the platform's own analytical layer or any other ecosystem.

Because every random draw in Stochast is deterministic given a fixed `Seed` value, the exported CSV is reproducible: another student running the same patch with the same seed will export a byte-identical CSV.

Troubleshooting

No sound / no signal flow. Press the **Play** button at the top of the Rack window. Without play, no audio (and no CV) processes.

Modules don't appear in the browser. Confirm the `.vcvplugin` archive sits in the correct plugin folder for your platform, then restart Rack completely (Cmd-Q on macOS — closing the patch alone is not enough).

A patch loads with empty viz areas. The module is processing but no data has reached it yet. Make sure the CLOCK input is patched (or rely on the internal ~30 Hz clock) and that the upstream module is producing CV.

Two clones of the same patch produce slightly different results. Check that every random module's RESET input is patched to the same Seed module's TRIG output. Without explicit reset wiring, the RNG

state at module-construction time is consistent, but any subsequent randomization buttons or right-click ☐ Initialize operations will desynchronise the streams.

Knob value isn't precise enough. Right-click any knob and choose *Type value...* to enter a numerical value directly.

Further reading

The accompanying paper, *Stochast: A Modular-Synthesizer Platform for Simulation-Based Statistics Education*, develops the pedagogical argument and places the platform in the context of the GAISE / Cobb / Tintle simulation-based-inference literature, the constructionist heritage of Papert and Resnick, and the older tradition of physical statistical instruments.

For implementation-level documentation (source structure, algorithms, numerical conventions), see the project README and the inline header comments of each `.cpp` file in `Methods/src/`.